Software Requirements Specification: Real-Time Lab Monitoring & Attendance System

Oct 14, 2026



Division of Computer Science and Engineering, Karunya Institute of Technology and Sciences

Course: 23CS2048 Web Technology, 3rd Internal Assessment (Project)

Team NULL

Name	Roll number	Role	Owns
A M Armaan	URK24CS1021	Team lead, backend and architecture	Apps Script server, Sheets data model, scan verification, triggers
Pooja Shree Ravichandar	URK24CS1277	Frontend and dashboards	Student app, kiosk and scanner screens, faculty, staff and admin dashboards, Karunya styling, report templates
Jonathan Jerald	URK24CS1112	Testing, security and reports	Attack testing, load simulation, real-phone testing, inventory module, report exports

1. Introduction

1.1 Purpose

This document describes what we're building for the Real-Time Lab Monitoring, Geofenced Attendance and Asset Booking System, how it works, and how we'll test it. It's written for the evaluators and for us, so we all agree on what "done" means before we start coding.

1.2 Why there are two versions

While planning, we walked through real lab sessions and tried to break the system the way a student would. Two things came out of that.

First, the spec as written has a few rough edges, like a QR token that only one person can use at a time while 60 people arrive together. Those are easy to fix without changing anything the spec asks for.

Second, and more importantly: the geofence relies on the phone telling the server where it is. Anyone who opens the browser console can send the lab's coordinates from their hostel room. No browser can detect that.

So this SRS has two parts:

- **Version A** builds everything in the Functional Requirements document, plus the fixes we found that don't change anything it asks for. It's honest about what GPS can and can't prove.
- **Version B** is what we'd build with a free hand. Attendance proof comes from a device that's actually in the room, not from the student's phone. It costs nothing extra and needs no GPS.

Both versions share the same stack, database, dashboards, inventory and reports. The only real difference is how a student checks in. We'll build Version A in full, and we plan to ship Version B's check-in as a mode that can be switched on.

1.3 Scope

In scope: attendance with check-in and check-out, live occupancy, hardware inventory and bookings, role-based dashboards, audit logs, and daily, monthly and custom reports.

Out of scope: native Android or iOS apps, biometrics, paid hardware or services, and attendance for non-lab classes.

1.4 References

- Circular KITS/CSE/CIR/2026/163, 12 October 2026
- Functional Requirements document attached to the circular
- Google Apps Script quotas and limits

1.5 Terms

Term	Meaning here
TOTP	A code that changes every few seconds, generated from a shared secret and the current

Term	Meaning here
	time
Geofence	A circle around the lab; a check-in counts only if the phone reports a position inside it
Haversine	The formula for distance between two GPS points on a sphere
Nonce	A random value the server hands out that can only be used once
ECDSA key pair	A private key that signs and a public key that verifies; the private key never leaves the phone
RBAC	Role-based access control: what you can do depends on your role
CR	Class representative
Kiosk	The lab monitor that shows the rotating QR (Version A)
Scanner	The teacher's or CR's device that scans students' QR codes (Version B)

2. Overall description

2.1 Users and roles

Role	What they do	Versions
Student	Check in and out, pick a visit reason, see the live lab count, request equipment, see their borrowing history	A, B
Faculty	See the live roster for their session, verify attendance, run spot checks, export session logs	A, B
Class rep	Session helper: starts or ends a session if needed, triggers a rescan when the teacher asks, and in Version B can scan classmates	A (helper), B (helper + scanner)
Lab incharge / staff	Watch live lab activity and occupancy, manage inventory,	A, B

Role	What they do	Versions
	issue and take back equipment, approve phone changes (B)	
Super admin	Set GPS coordinates, geofence radius and hours, manage roles, read audit logs	A, B

2.2 Operating environment

- **Students:** any smartphone with a recent Chrome, Safari or Samsung Internet, logged into their Karunya Google account. Nothing to install.
- **Version A kiosk:** an existing lab PC and monitor running the kiosk page full-screen.
- **Version B scanner:** the teacher's phone or laptop, or the CR's phone.
- **Network:** campus Wi-Fi or mobile data.
- **Backend:** Google Apps Script on the university's Google Workspace.

2.3 Constraints (from the spec)

- 100% Google Apps Script and Workspace, zero server cost
- Backend on the V8 runtime with doGet/doPost endpoints
- Single-page frontend through HtmlService (HTML5, Tailwind CSS, JavaScript)
- Google Sheets as the database
- DriveApp, DocumentApp and GmailApp for storage, reports and mail
- PropertiesService for configuration

2.4 Assumptions

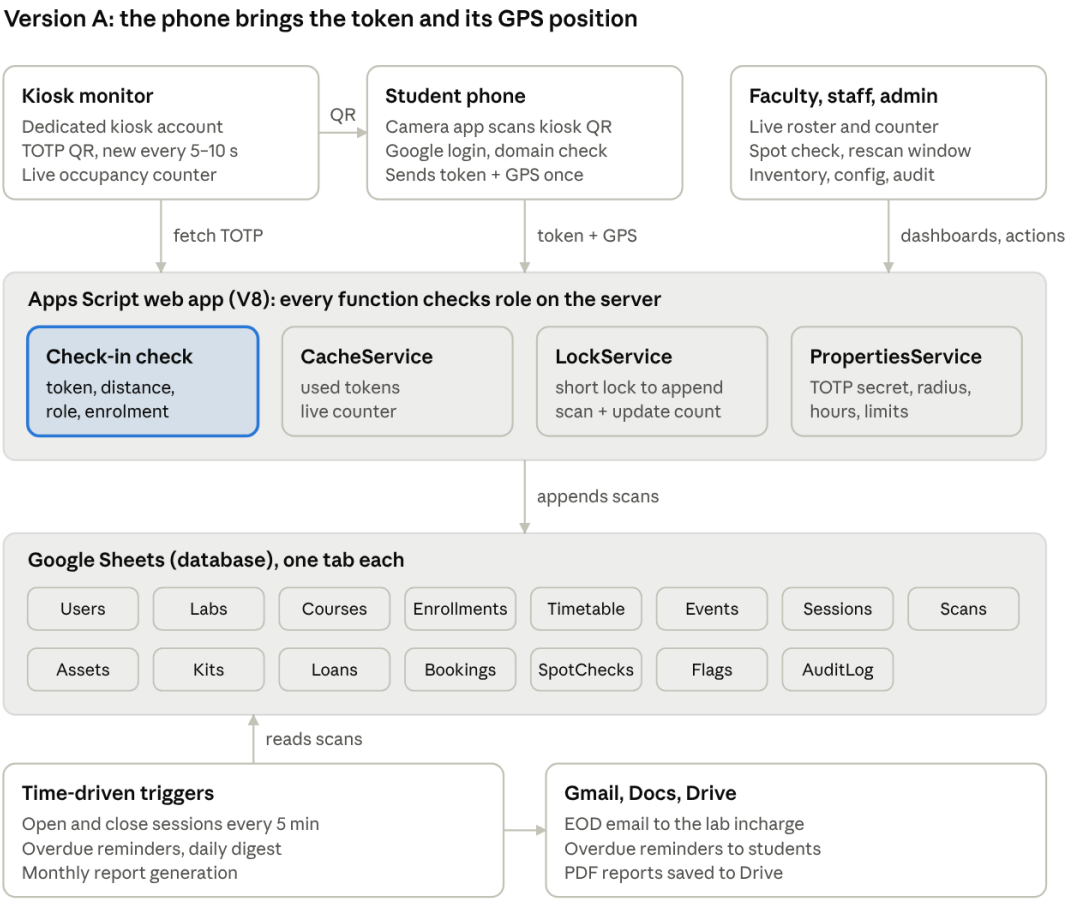
- Everyone has a Google Workspace for Education account, so Workspace quotas apply.
- @karunya.edu and @karunya.edu.in may be separate domains. We check the domain ourselves on the server instead of relying on the deployment's "same domain only" setting.
- Students have a smartphone with data or Wi-Fi in the lab.
- Each lab already has a PC and monitor we can use as a kiosk (Version A).

2.5 Dependencies and risks

- **Browser permissions inside Apps Script.** HtmlService pages run inside Google's iframe, and Google controls what that iframe allows. Developers reported in February 2026 that camera access stopped working in Apps Script web apps, and it was still broken as of late April (Google developer forum). Geolocation could be affected the same way. We test both on day one; section 6.4 covers the fallbacks.
- **Google quotas.** Covered in section 14.

3. Version A: overview and architecture

Version A follows the spec exactly: a monitor in the lab shows a QR that changes every few seconds, students scan it with their phone camera, and the app checks the token and the phone's GPS position before checking them in.



Version A architecture · 3 clients, Apps Script services, 15 sheet tabs, triggers

Everything runs in one Apps Script project. The kiosk and dashboards poll the server every few seconds for live numbers. All writes go through the server, so no browser ever touches the sheet directly.

4. Version A: functional requirements

Everything the spec asks for is here. Rows marked **(added)** are our fixes; none of them remove or change a spec requirement.

4.1 Authentication and roles

ID	Requirement	Notes
FR-1	Only @karunya.edu and @karunya.edu.in Google	Checked on the server for every

ID	Requirement	Notes
	accounts can sign in	request
FR-2	First login collects name, roll number or employee ID, Karunya email and mobile number	
FR-3	Roll number is taken from the email where possible, locked after setup, and must be unique **(added)**	Stops students from typing a friend's roll number
FR-4	Four roles: Student, Faculty, Lab Incharge/Staff, Super Admin, with the permissions listed in the spec	
FR-5	Every server function checks the caller's role itself; internal helpers end in _ so the browser can't call them **(added)**	Hiding a button isn't security
FR-6	Only a Super Admin can change roles, and every change goes to the audit log	

4.2 Attendance engine

ID	Requirement	Notes
FR-7	The kiosk shows a TOTP QR that rotates every 5–10 seconds	Interval set in PropertiesService
FR-8	Scanning the QR with the phone camera opens the web app with the token	
FR-9	The token is checked the moment the page opens; the student then has 2 minutes to finish **(added)**	Login and GPS take longer than one QR rotation
FR-10	A token can be used once per student, not once globally **(added)**	Otherwise 60 students queue for about 7 minutes

ID	Requirement	Notes
FR-11	The app captures GPS and checks distance to the lab with Haversine against a 15–20 m radius	Radius and coordinates set by Super Admin
FR-12	The reported GPS accuracy is factored into the check, so a 30 m indoor error doesn't wrongly reject someone sitting in the lab **(added)**	
FR-13	The student must pick one entry reason: Regular Lab Course, Event/Workshop, or Individual Research, with the fields the spec lists	
FR-14	For a scheduled class, the course is pre-filled from the timetable and the student just confirms **(added)**	Avoids 60 different spellings of one course code
FR-15	Check-out uses the same scan; a minimum gap stops an accidental double scan from checking out **(added)**	
FR-16	Live occupancy goes up on check-in and down on check-out, shown on every dashboard and refreshed every few seconds	
FR-17	When a session ends, anyone who didn't check out is marked "unverified exit" and gets no hours **(added)**	Prevents scan-in-and-leave
FR-18	The system supports multiple labs, each with its own coordinates, kiosk, inventory and staff **(added)**	The circular says it will be rolled out across all specialised labs

4.3 Assets and bookings

ID	Requirement	Notes
FR-19	Inventory catalog: GPU workstations, IoT hardware, sensors and modules	
FR-20	Students can see what's available without checking in	
FR-21	Requesting equipment needs an active CHECKED_IN status	
FR-22	Staff confirm issue and return on a handshake screen	
FR-23	Overdue items are tracked, and reminders go out by email automatically	
FR-24	Kits (for example an "LED blink kit") can be issued as one item, in bulk to a whole batch, to pairs or groups **(added)**	One-item-at-a-time is 360 handshakes for a 60-student class
FR-25	GPU workstations can be booked in time slots in advance; a no-show releases the slot after 15 minutes **(added)**	The spec's check-in gate makes advance booking impossible otherwise
FR-26	Boards are tracked by serial number; consumables like jumper wires by count **(added)**	

4.4 Security and anti-spoofing

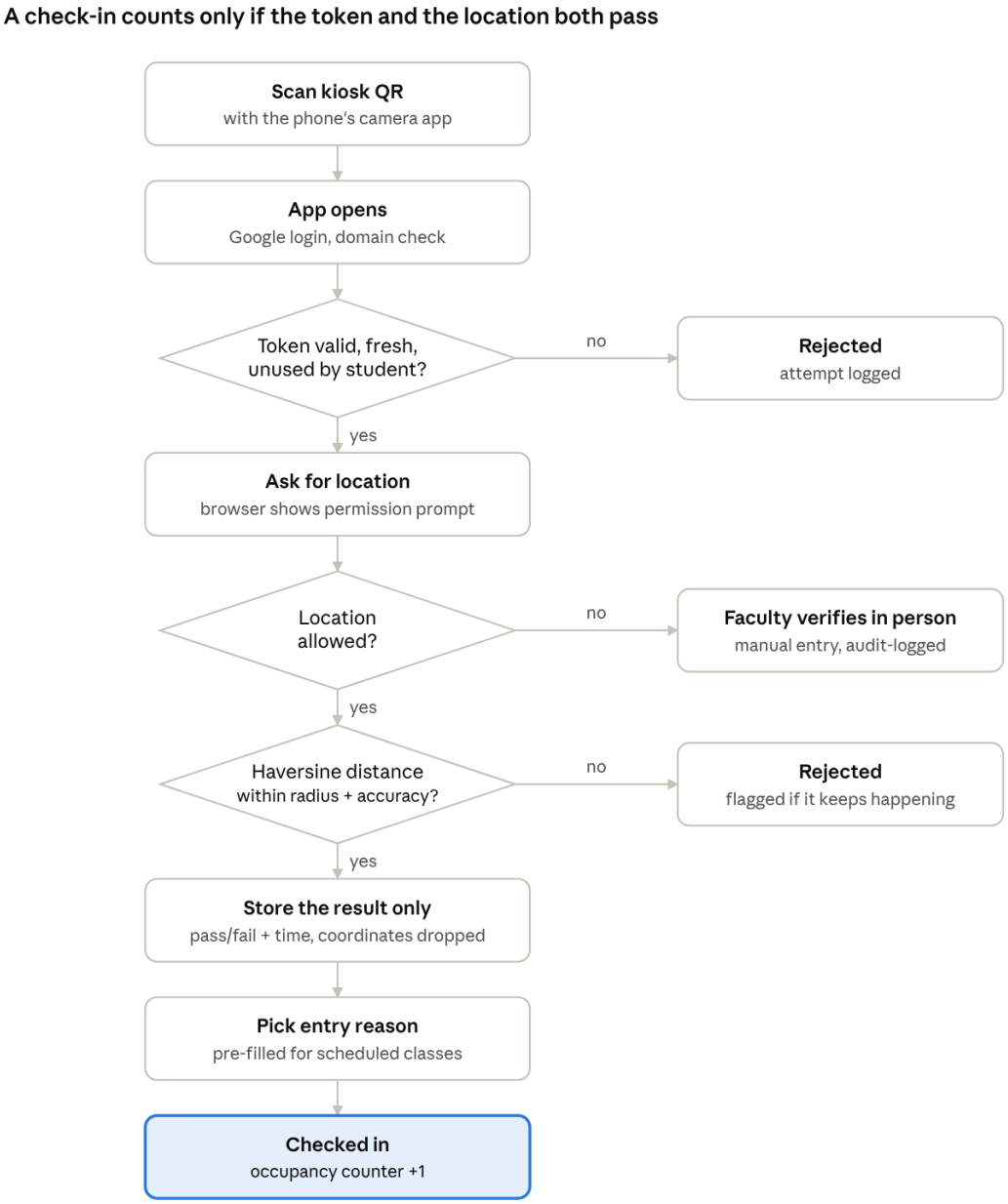
ID	Requirement	Notes
FR-27	Used tokens are stored in CacheService and rejected if reused	
FR-28	Check-in writes run under LockService; only the row append and counter update sit inside the lock	Keeps the lock short at peak
FR-29	Impossible-travel check	Meaningful only between

ID	Requirement	Notes
	between consecutive check-ins	different labs (see 6.3)
FR-30	Mock-location heuristics: reject readings with impossible accuracy, and flag identical readings across sessions	Browsers expose no mock-location flag (see 6.3)
FR-31	Only `is_valid_geofence` and a timestamp are stored; raw latitude and longitude are discarded after the check	
FR-32	Append-only audit log of manual entries, role changes and inventory overrides; each row stores a hash of the previous row **(added)**	Makes editing old rows detectable
FR-33	The kiosk runs under a dedicated account that can only display QR codes **(added)**	A student at the kiosk gets nothing useful

4.5 Analytics and reporting

FR-34 to FR-39 cover the daily digest, monthly audit, custom range export, PDF via Docs template, CSV/Excel export and the EOD email. Details are in section 12.

5. Version A: geofencing workflow



Version A geofencing workflow · 3 checks, 3 exits

Raw coordinates never reach the sheet. The server works out the distance, keeps only pass or fail plus a timestamp, and throws the numbers away. Check-out follows the same path without the reason step.

6. Version A: what it protects, what we improved, and its limits

Version A stops casual cheating, like screenshots or checking in from far away without effort. It can't stop a student who knows how to use the browser console, and we'd rather say that here than have it found later.

6.1 How each spec security item is implemented

Spec item	How we do it
TOTP and single-use tokens	Server generates TOTP from a secret in PropertiesService; used tokens per student go into CacheService and are rejected on reuse
Concurrency protection	LockService around the row append and counter update only, so each lock is held for a fraction of a second
Impossible travel	Time and distance between a student's consecutive check-ins at different labs; anything faster than walking is flagged
Mock location	Heuristics only: accuracy values that are too perfect, and identical readings across sessions, get flagged
Data privacy	Only `is_valid_geofence`, distance band and timestamp are stored; raw coordinates are dropped after the check
Audit trail	Append-only sheet, each row carrying a hash of the previous row, sheet protected and shared only with the owner account

6.2 Improvements we added

All of these sit inside the spec without changing it:

1. Tokens are single-use per student, so a whole class can scan the same QR.
2. The token is validated when the page opens, followed by a 2-minute window to finish.
3. GPS accuracy is factored into the geofence check.
4. Course is pre-filled from the timetable.
5. Anyone who doesn't check out gets an "unverified exit" instead of free hours.
6. Roll number is locked to the account.
7. Every server function checks roles itself.
8. The kiosk uses a dedicated low-privilege account.
9. The audit log is hash-chained.
10. Multi-lab support from day one.
11. Kits, bulk issue, groups and tiered tracking for inventory.

12. Time-slot booking for GPU workstations.
13. Faculty spot checks: the app shows 5 random present students' photos and the teacher taps present or absent.
14. Faculty rescan window: everyone must rescan within 90 seconds, at a time the teacher picks.

6.3 Honest limits

- **GPS can be faked.** The phone sends its coordinates to the server as plain numbers. A student can call the check-in function from the browser console with the lab's coordinates and accuracy of 5 m. No server or browser check can tell. Spot checks (item 13) and rescan windows (item 14) are what catch this.
- **Mock-location detection isn't possible in a browser.** Only native Android apps can see that flag. Our heuristics catch lazy spoofing, not careful spoofing.
- **Impossible travel only works between labs.** With one lab, every valid check-in is at the same point, so speed can't be calculated. Raw coordinates are also discarded by design, so we only compare lab-to-lab hops.
- **Account sharing.** If a student gives a friend in the lab their login, the friend can check in for them. Spot checks are the only defence.
- **Indoor GPS is often off by 20–50 m.** Factoring in accuracy reduces false rejections, but a few will still happen, so faculty need a manual-verify option (logged).

6.4 Platform risk and fallbacks

Apps Script web apps run inside a Google iframe, and Google decides which browser features it allows. Camera access in Apps Script web apps was reported broken from February 2026 (forum thread), and geolocation could be affected the same way.

- **Camera:** Version A doesn't need it. Students scan the kiosk QR with their phone's normal camera app, which opens our web app.
- **Geolocation:** we test it on day one on Android Chrome and iPhone Safari. If it's blocked, Version A still works on the rotating QR alone, the geofence result is recorded as "not available", and we tell the evaluators. Version B doesn't need GPS at all.

7. Version B: our design

Version B flips the scan around. Instead of the student's phone reading a QR on the wall and reporting its location, the student's phone shows a QR, and a device that's actually in the room (the teacher's or CR's) scans it. The proof of presence comes from the scanner, so GPS isn't needed and faking coordinates gets you nothing.

7.1 The device key

The obvious idea is to tie each student to their phone's MAC address. That doesn't work: browsers can't read MAC addresses, IMEI or any hardware ID, and phones randomise their Wi-Fi MAC anyway. So we use a cryptographic key instead.

1. **Registration (once):** the browser creates an ECDSA key pair with Web Crypto, marked non-extractable. The private key stays inside that phone's browser storage; even dev tools can't export it. The public key is saved against the student in the Devices sheet.
2. **First phone is approved automatically.** Switching phones needs lab staff to approve it in person, and the old key is revoked. One active phone per student.
3. **Server nonce:** the app fetches a fresh random value from the server every 30–60 seconds.
4. **The QR:** every 2–3 seconds the phone signs `{student ID, nonce, timestamp, counter}` and shows the result as a QR.

Because the nonce comes from the server and the server only accepts the latest two, nobody can pre-generate a stack of QR codes at home by changing the phone's clock.

7.2 Who scans, and how (zero cost)

The scanner is the teacher's phone or laptop, or the CR's phone. No extra hardware.

- **Phone as scanner (main method):** each student QR holds a link to our web app with the signed payload attached. The teacher or CR points their phone's normal camera app at it, taps the link, and the app opens already logged in as them. The server checks they're allowed to scan, verifies the payload, and shows the student's name, photo and "IN" or "OUT". About 2–4 seconds per student. With the teacher and CR scanning together, a 60-student class takes about 2–3 minutes.
- **Laptop webcam (optional):** camera access inside Apps Script is currently blocked (section 6.4), so this needs a tiny scanner page hosted outside Apps Script, for example on GitHub Pages for free. It reads the QR through the webcam and opens the same link. It breaks the "100% Apps Script" rule, so it's an extra, not a requirement.
- **If a student scans another student's QR with their own phone,** the link opens as a student account, and the server refuses it.

Class rep scanning comes with limits, because a CR's phone can leave the room:

- Only during an active session, and only for that class's roster
- The permission ends automatically when the session ends
- A CR can't scan themselves
- Every CR scan is tagged, so the teacher sees who scanned whom

Self-work visits with no teacher around (evenings, research): lab staff do the scanning. If no staff member is present, the visit falls back to the Version A kiosk flow and is marked as lower-assurance.

7.3 During and after class

- **Rescan window:** the teacher can ask everyone to rescan within 90 seconds at any point. Anyone who checked in and left gets caught.
- **Spot check:** the app shows 5 random present students' photos, and the teacher taps present or absent.
- **Exit scan:** needed for hours to count. No exit scan means "unverified exit".

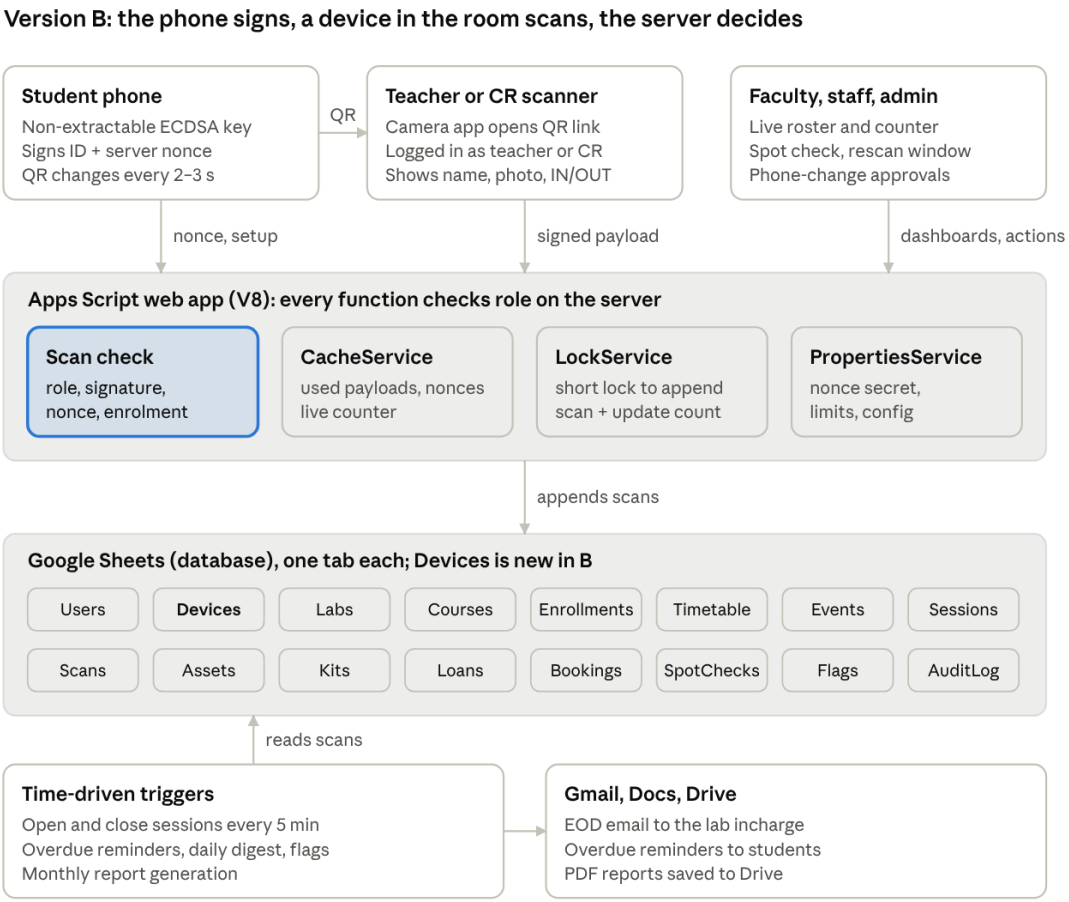
7.4 Verifying signatures in Apps Script

Apps Script's built-in `Utilities` can create RSA signatures but can't verify ECDSA ones, so we paste a pure-JavaScript crypto library into the project, most likely jsrsasign, which supports ECDSA verification. We'll use the latest release, since older versions had signature-checking bugs. Two details matter:

- We benchmark verification on day one, since it runs on every scan.
- We mark the **payload** (the nonce and counter) as used, not the signature itself. ECDSA signatures can be tweaked into a different but still valid form, so checking the signature string alone would let a copied QR through twice.

8. Version B: architecture and scan flow

Same backend as Version A, with three changes: a Devices tab for public keys, a scan check that verifies signatures instead of GPS, and a scanner role in place of the kiosk.



Version B architecture · student phone, scanner, dashboards, 16 sheet tabs

What happens on each scan:

1. The student opens the app; their QR is already showing and changing every 2–3 seconds.
2. The teacher or CR points their camera at it and opens the link.

3. The server checks, in order:
    a. the scanner is a teacher, staff member, or a CR during their own session
    b. the student's device is active and the signature matches its public key
    c. the nonce is one of the latest two and the timestamp is fresh
    d. this payload hasn't been used before
    e. the student is on the roster for this session
4. If all pass, the server flips the student IN or OUT, appends the scan, and updates the counter. Only this step runs inside the lock.
5. The scanner screen shows the student's name, photo and IN or OUT, so a mismatch is obvious to anyone watching.

9. Version B: proxy attempts, how we stop them, and what still gets through

Every remote way of faking attendance is blocked. What's left needs someone physically in the room helping, real effort, or a staff member in on it.

9.1 Blocked or caught

Proxy attempt	How it's stopped
Fake GPS or typed coordinates	GPS isn't used; presence is proven by the scanner in the room
Sending a screenshot of your QR	QR changes every 2–3 seconds and each payload works once
Pre-generating QRs by changing the phone clock	Server nonce signed into every QR; only the latest two are accepted
Generating a friend's QR on your phone	The signing key exists only on the friend's registered phone
Logging into a friend's account on your phone	No approved key on that phone; changing phones needs in-person staff approval
Typing a friend's roll number at setup	Roll number comes from the email or is locked; must be unique
Scanning in, then leaving	Random rescan windows; exit scan needed for hours
Never scanning out	Marked "unverified exit", no hours
Calling server functions from the console	Every function checks role on the server
CR scanning friends outside the lab	Scans only during the session, only that roster, all tagged for the teacher
"My phone isn't working, sir"	Manual marking only for a student standing in

Proxy attempt	How it's stopped
	front of the teacher; frequent use flagged
Spamming scans to force manual marking	Rate limit per student
Editing the Sheet directly	Shared only with the owner account; hash-chained audit log

9.2 What it can't fully stop

Bypass	What limits it
Handing your phone to a friend, who gets scanned twice	Scanner shows each student's photo; back-to-back scans from one person get flagged; spot checks
Live video of your phone screen held up by a friend	Awkward with a QR changing every 2–3 seconds; spot checks
Scanning in, leaving, coming back to scan out	Caught only if a rescan window happens while they're gone
Talking staff into approving a "new phone"	In-person approval, old phone revoked, logged
Copying a laptop browser profile to move the key	Low risk since students register on phones; laptop registrations are flagged
A CR, teacher or admin colluding	No software fixes this; the audit log makes it visible

Spot checks make cheating a gamble: checking 5 of 60 students per session over 12 sessions catches a regular proxy about 65% of the time per semester. That only works as a deterrent if being caught has a real consequence.

10. Data model

One Google Sheet, one tab per table. Every tab has an ID column as its first column, and other tabs refer to rows by that ID. Version B adds one tab (Devices); everything else is shared.

Tab	Key columns	Notes
Users	user_id, email, name, roll_or_emp_id, mobile, role, photo_file_id, status	Roll number unique and locked
Devices (B)	device_id, user_id, public_key, registered_at, status, approved_by	One active device per student

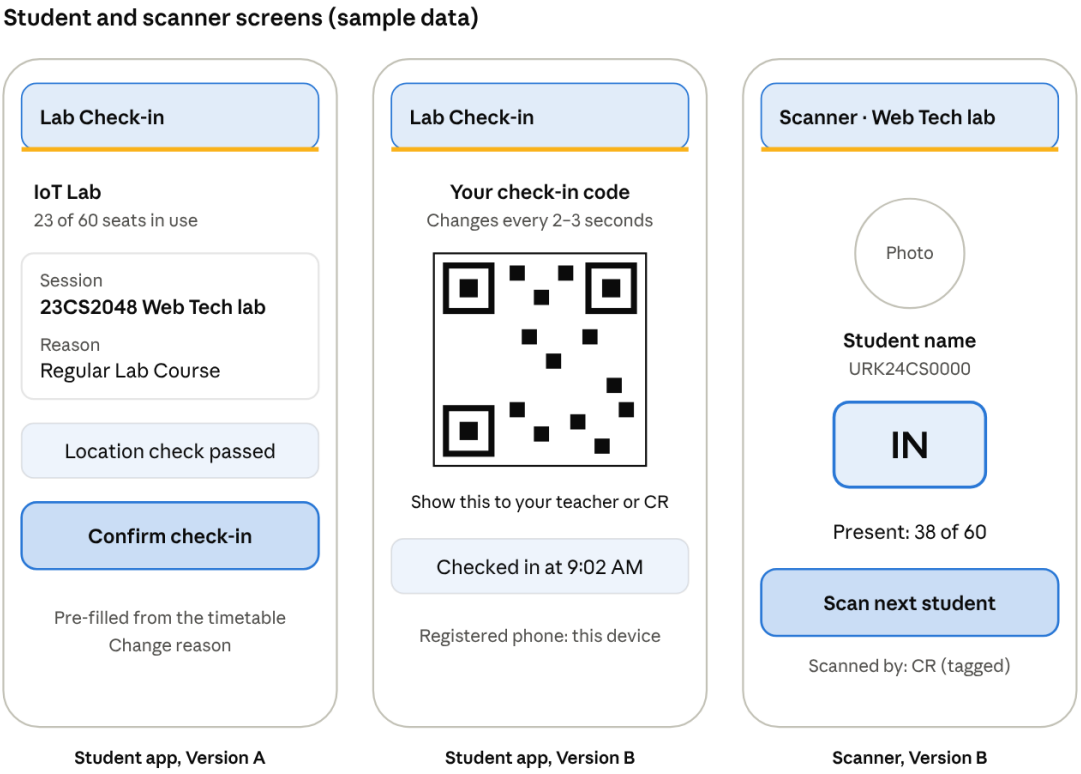
Tab	Key columns	Notes
Labs	lab_id, name, latitude, longitude, radius_m, open_time, close_time, capacity	Coordinates only for labs, never for students
Courses	course_code, course_name, faculty_id	
Enrollments	course_code, user_id, batch, group_no	Roster for each class
Timetable	slot_id, lab_id, course_code, weekday, start, end, faculty_id	Drives session open and close
Events	event_id, name, coordinator_email, lab_id, start, end, registration_list	Workshops and hackathons
Sessions	session_id, lab_id, type (course/event/self-work), ref_id, opened_at, closed_at	One row per running session
Scans	scan_id, session_id, user_id, direction (in/out), time, method (kiosk/scanner/manual), scanned_by, is_valid_geofence (A), reason, flags	Append-only
Assets	asset_id, category, name, serial_no, lab_id, status, condition	Boards and workstations
Kits	kit_id, name, contents, lab_id, status	Bundles like an LED blink kit
Loans	loan_id, asset_or_kit_id, user_or_group_id, issued_by, issued_at, due_at, returned_at, return_condition	Issue and return handshakes
Bookings	booking_id, asset_id, user_id, slot_start, slot_end, status	GPU workstation time slots
SpotChecks	check_id, session_id, user_id, result, checked_by, time	
Flags	flag_id, user_id, type, details, time, resolved_by	Anomalies for faculty review

Tab	Key columns	Notes
AuditLog	log_id, time, actor, action, target, before, after, prev_hash, hash	Append-only, hash-chained

Configuration (TOTP secret, nonce secret, token interval, rate limits, default radius) lives in PropertiesService, not in the sheet. Live state (used tokens and nonces, occupancy counters, active sessions) lives in CacheService and gets rebuilt from the Scans tab if the cache is cleared.

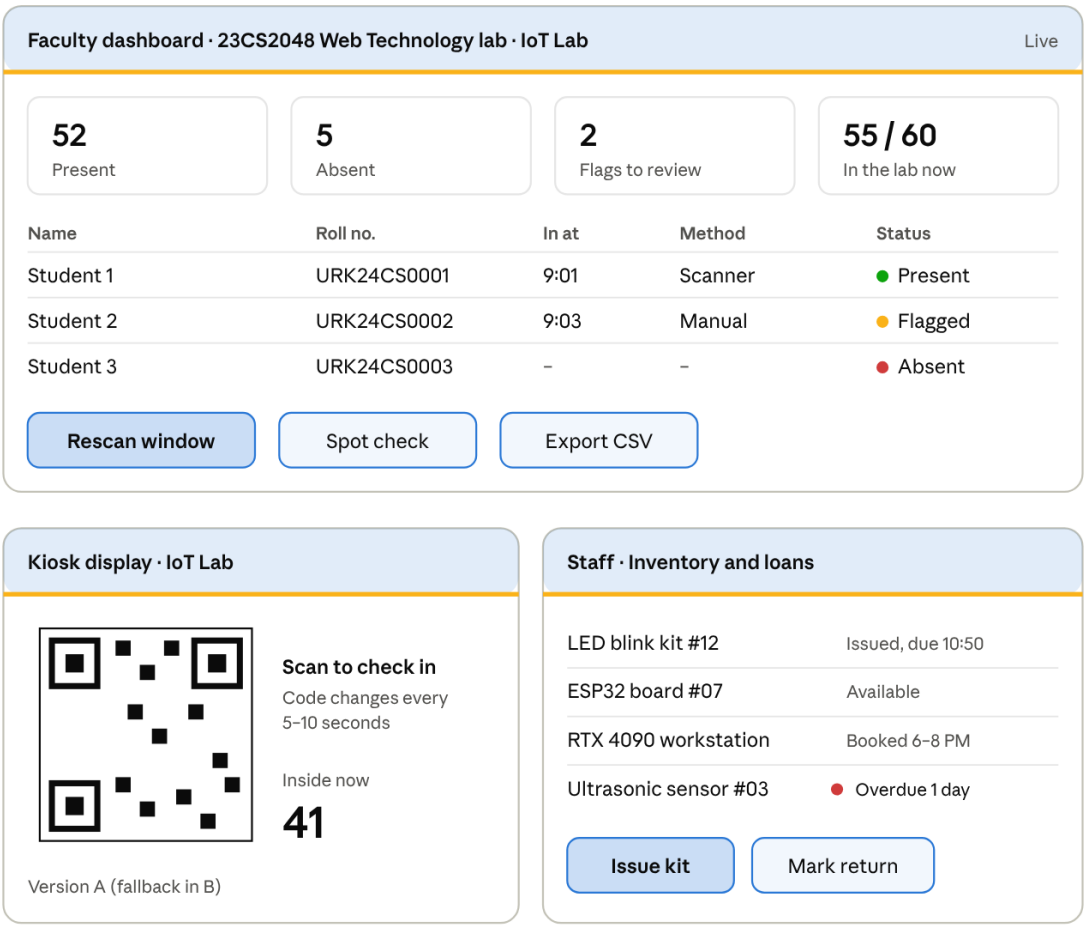
11. Dashboard designs

These are layout mock-ups with sample data. The built app uses Karunya blue for headers and buttons, with a thin gold rule under each header, matched to the logo colours. All screens are mobile-first, except the faculty dashboard and kiosk, which are designed for a laptop or monitor.



student and scanner screens · sample data

Faculty, kiosk and staff screens (sample data)



faculty, kiosk and staff screens · sample data

Screen	Who uses it	Main things on it
Student app	Students	Check-in (A) or rotating QR (B), live lab count, equipment requests, borrowing history
Kiosk display	Lab monitor (A)	Rotating QR, lab name, live count
Scanner	Teacher, CR, staff (B)	Last scanned student's name, photo, IN or OUT, present count
Faculty dashboard	Faculty	Live roster, counters, flags, rescan window, spot check, export

Screen	Who uses it	Main things on it
Staff dashboard	Lab incharge, staff	Live activity feed, occupancy, inventory, issue and return, overdue list, phone-change approvals (B)
Admin panel	Super admin	Labs and coordinates, radius, hours, roles, quota usage, audit log viewer

12. Reports and analytics

All reports are built from the Scans, Loans and Bookings tabs, so they come out the same in both versions.

ID	Report	What's in it	When	Format	Goes to
FR-34	Daily digest	Unique visits, peak occupancy hour, hourly distribution, unreturned items, unverified exits, flags	Every evening, by time trigger	Email + PDF	Lab incharge
FR-35	Monthly audit	Student-hours per course, GPU workstation utilisation, sensor turnover, spot-check results	1st of each month	PDF in Drive	HOD, lab incharge
FR-36	Custom range	Any start and end date, for workshops, hackathons or assessment weeks	On demand	PDF or CSV	Whoever requests it
FR-37	Session log	Who was present, times,	After each	CSV/Excel	Faculty

ID	Report	What's in it	When	Format	Goes to
		method (kiosk, scanner or manual), flags	session		
FR-38	PDF generation	A Google Docs template filled by DocumentApp, converted to PDF and saved in a Drive folder	Used by FR-34 to FR-36	PDF	Drive
FR-39	ERP export	Raw rows in a fixed column order	On demand	CSV/Excel	Admin

Hours are always worked out from actual scan times, never from durations students type in. GPU utilisation comes from booked slots that were actually used.

13. Non-functional requirements

Area	Requirement
Speed at peak	A 60-student class checks in within about 3 minutes (A: everyone scans the kiosk at once; B: teacher and CR scanning together at 2–4 seconds per student)
Live updates	Occupancy and rosters refresh every 5 seconds by polling (Apps Script has no websockets)
Phones	Works on a mid-range Android phone and an iPhone in the default browser, nothing to install
Privacy	No raw GPS stored (A); no GPS collected at all (B). First login shows a short notice explaining what is collected and why, in line with India's DPDP Act
Security	Roles checked on the server for every call; secrets only in PropertiesService; sheet shared only with the owner account
Reliability	If the cache is wiped, counters and used tokens rebuild from the Scans tab; a failed scan shows a clear retry message instead of hanging

Area	Requirement
Usability	Check-in in under 10 seconds for a returning student; big readable text on the kiosk and scanner screens; Karunya blue and gold styling
Maintainability	Lab coordinates, radius, hours, token interval and limits are changed from the admin screen, not in code
Scale	Multi-lab from the start; stays inside Google Workspace quotas (section 14)

14. Cost

Both versions cost ₹0. Everything runs on Google services the university already has, and we only use devices that already exist.

Item	Version A	Version B
Apps Script, Sheets, Drive, Docs, Gmail	₹0	₹0
Kiosk	Existing lab PC and monitor	Not needed (optional fallback for self-work visits)
Scanner	Not needed	Teacher's or CR's phone, or teacher's laptop
Laptop scanner page (optional)	Not needed	Free static hosting such as GitHub Pages
Student phones and data	Students' own	Students' own
Staff time	A few minutes a day for kit issue and returns	Same, plus approving the occasional phone change

Google limits that shape the design (Workspace accounts, from Google's quota page):

Limit	Value	What it means for us
Simultaneous executions	30 per user	If the app runs as the owner, all requests likely share this. Keep each call short and dashboard polling light. Version B helps, since scans arrive one at a time per scanner
Email recipients	1,500 a day (2,000 within the	Plenty for digests and reminders; one combined

Limit	Value	What it means for us
	domain)	reminder per student per day
Trigger runtime	6 hours a day	Session open/close every 5 minutes plus reports fits easily if each run is short
Triggers	20 per user per script	A few dispatcher triggers, not one per session
Script runtime	6 minutes per run	Monthly reports are built in batches
Properties storage	500 KB	Configuration only
Documents created	1,500 a day	PDF reports are well inside this

Google can change these limits without notice, so the admin screen will show how much of the daily quota has been used.

15. Testing and demo plan

We test the way the system will actually be used and abused, on real phones, and we'll show the evaluators the results.

15.1 Day-one platform checks

- GPS inside the Apps Script page on Android Chrome and iPhone Safari
- Camera inside the Apps Script page (expected to be blocked; confirms which fallback we use)
- ECDSA key creation and storage on both phones, including after closing and reopening the browser (B)
- How long one signature verification takes in Apps Script (B)

15.2 Functional tests

Each FR in sections 4 and 12 gets at least one test case, tracked in a sheet with expected result, actual result and pass/fail.

15.3 Load test

A script fires 60 check-ins within a minute at the deployed app and records how many succeed, how long each takes, and whether any lock or quota errors appear. We repeat it with dashboards polling at the same time.

15.4 Attack tests

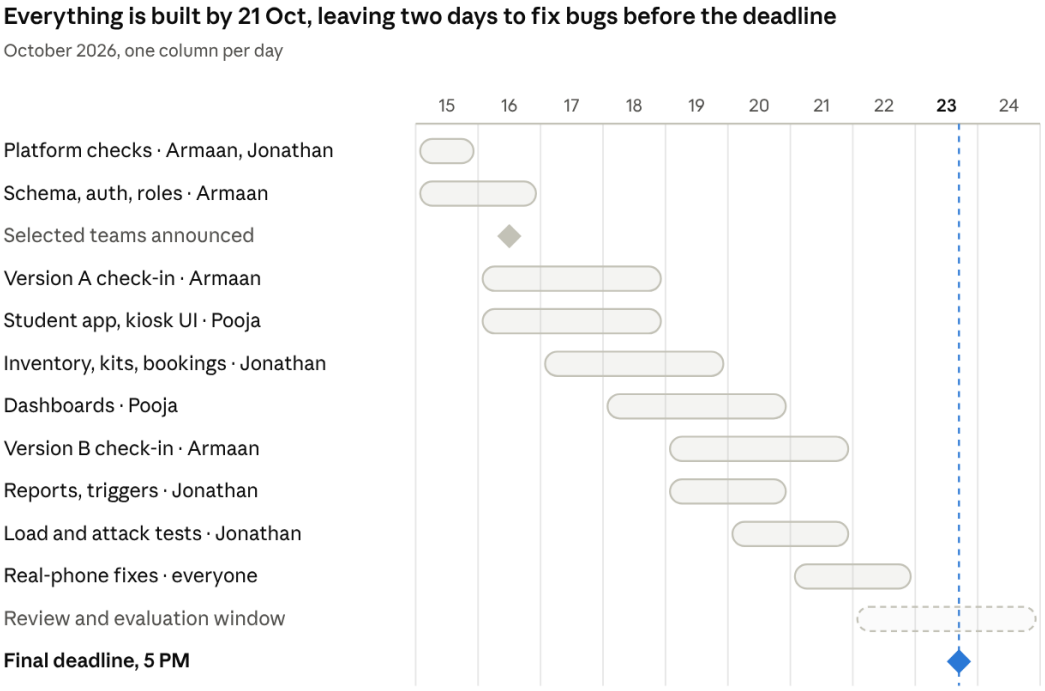
Attack	Expected result
Calling check-in from the console with the lab's coordinates	A: accepted (documented limit); B: not applicable

Attack	Expected result
Reusing a scanned token or payload	Rejected
Screenshot of a QR sent to a friend	Rejected after expiry
QR generated with the phone clock moved ahead (B)	Rejected, stale nonce
Calling an admin function as a student	Rejected
Student scanning another student's QR (B)	Rejected, not a scanner role
CR scanning after the session ends (B)	Rejected
Editing a past AuditLog row by hand	Detected by the hash check

15.5 Demo for evaluation

1. A student registers and checks in on a real phone (both versions).
2. Teacher dashboard updates live; spot check and rescan window shown.
3. Kit issued and returned; overdue reminder triggered.
4. Attack demos from 15.4, live.
5. Load test results and a generated PDF report.

16. Build timeline



build timeline · 15–24 October 2026, owner per task

We start on 15 October, before selection is announced, so we aren't squeezed if we're picked. Version A comes first because it's the requirement; Version B's check-in reuses everything else and is built on top. If something slips, Version B's laptop scanner page is the first thing we drop.

Appendix A: Spec gaps and how we resolved them

#	What the spec says	The problem	What we did
1	Tokens are single-use and rotate every 5–10 s	One check-in per rotation means about 7 minutes for 60 students	Single-use per student (FR-10)
2	Token rotates every 5–10 s	Login and GPS take longer, so tokens expire mid check-in	Validate on page open, 2-minute window (FR-9)
3	Geofence radius 15–20 m	Indoor GPS is often off by 20–50 m	Factor in reported accuracy (FR-12)
4	Impossible-travel check, plus discard raw coordinates	Can't compute speed without coordinates; with one lab every check-in is at the same point	Compare lab-to-lab hops only (FR-29)
5	Mock location detection	No browser exposes this	Heuristics, documented as a limit (FR-30, 6.3)
6	Coordinates of "the IoT lab" but deployed in all labs	Single-lab design	Multi-lab from the start (FR-18)
7	Bookings need CHECKED_IN status	GPU workstations can't be booked in advance	Time-slot bookings (FR-25)
8	Check-out on sign-out	Students forget; counter inflates	Unverified exit at session end (FR-17)
9	Profile takes a typed roll number	A friend's roll number gets the attendance	Locked to the email (FR-3)
10	No course list, timetable, roster or events	Free-text course codes, no way to know who's absent	Courses, Enrollments, Timetable and Events tabs (section 10)
11	Items checked out one at a time	360 handshakes for one 60-student lab	Kits, groups, bulk issue (FR-24)

#	What the spec says	The problem	What we did
12	Append-only audit on Sheets	The owner can still edit rows	Hash-chained rows (FR-32)
13	Location stored as a boolean	Still needs a consent notice	DPDP notice on first login (section 13)
14	Doesn't mention the iframe Apps Script runs in	Camera (and possibly GPS) access can be blocked by Google	Day-one tests and fallbacks (6.4)

Appendix B: Scenario walkthroughs

We walked through three real situations step by step to find where things break. Timings are our estimates.

B.1 Regular lab class: 60 students, LED blink experiment

When	What happens	What broke	Fix
8:55	First-time students arrive	Profile setup at the door blocks the queue	Onboarding before the first lab
9:00	Everyone scans at once	Global single-use token, token expiry mid-flow, lock timeouts	FR-9, FR-10, short locks (FR-28)
9:00	GPS check	10–20 students rejected by indoor GPS error	Accuracy factored in (FR-12); Version B skips GPS
9:00	Picking a reason	60 spellings of one course code	Timetable pre-fill (FR-14)
9:10	Kits handed out	One item at a time is 360 handshakes; LEDs and jumpers aren't in the catalog	Kits, groups, bulk issue (FR-24, FR-26)
9:40	Burnt LED, faulty board	No swap or damage flow	Return condition and swap on the loan record
10:45	Everyone leaves and returns kits at once	Jumper wires can't be counted in 5 minutes	Quick condition checklist, shortfalls settled later
10:50	20 students never	Counter shows about	Session close,

When	What happens	What broke	Fix
	scanned out; next batch arrives	110 in a 60-seat lab	unverified exit (FR-17)

B.2 Saturday workshop: 80 participants, 9 AM to 4 PM

What happens	What broke	Fix
Participants come from several years and departments	They aren't on any course roster	Events created in advance with a registration list (Events tab)
The coordinator is a student or an outside speaker	Not a faculty account, so can't run the session	Temporary "event coordinator" permission, limited to that event and expiring with it
A guest speaker without a Karunya account	Domain lock blocks them	Out of scope; the coordinator adds them manually, logged
80 people, 60 seats	Lab over capacity	Capacity warning on the coordinator dashboard
Everyone leaves for lunch	80 check-outs and check-ins in a few minutes	Lunch break set in the event, so attendance pauses instead of needing scans
Certificates need minimum hours	Hours must be trustworthy	Hours computed from scan times; custom range report (FR-36)
Kits for 40 pairs	Same handshake problem as B.1	Bulk issue to groups (FR-24)

B.3 Solo GPU visit: research student at 6 PM

What happens	What broke	Fix
Wants an RTX 4090 workstation tonight	Spec only allows requests after checking in	Book a 2-hour slot in advance (FR-25)
Arrives 20 minutes late	Slot sits unused	No-show releases the slot after 15 minutes
No teacher or CR around (Version B)	Nobody to scan	Lab staff scan; if none present, kiosk fallback marked lower-assurance (7.2)
Lab closes at 7 PM, student stays	No closing-time handling	Overstay alert to staff at closing time

What happens	What broke	Fix
Leaves without checking out	Workstation looks busy and utilisation is wrong	Booking ends automatically at slot end; unverified exit
Monthly report	Utilisation needs real usage	Calculated from slots actually used, not just booked

Appendix C: Placeholders and open items

These get filled in once we have the details. None of them change the design.

Item	Placeholder	Who confirms
Labs covered in the first rollout	[Lab names]	Division / lab incharges
Capacity of each lab	[Seats per lab]	Lab incharges
IoT lab GPS coordinates	[Latitude, longitude]	Measured on site
Geofence radius	20 m to start, tuned after testing	Super admin
Lab operating hours	[Open and close times]	Lab incharges
Are @karunya.edu and @karunya.edu.in one Google Workspace?	Treated as separate until confirmed	IT department / Dr. I. Titus
Kiosk PC per lab (Version A)	Assumed available	Lab incharges
Karunya logo for reports and app header	[Logo file]	Team
Exact brand colours	Karunya blue and gold, matched to the logo	Team